Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

EXPERIMENT NO. 3

Deploying Multi-Service Applications Using Docker Compose

LO4: Containerization, orchestrate and deploy applications using Docker, Compose, Kubernetes and Cloud infrastructure tools.

1. Aim

To understand the concept of multi-container orchestration and to deploy a multi-service application consisting of a Nginx web server and a MySQL database using Docker Compose, covering service definition, environment configuration, networking, and volume persistence.

2. Theory

What is Docker Compose?

Docker Compose is a tool provided by Docker for defining and running multi-container applications.
Instead of manually creating and linking multiple containers using separate 'docker run' commands,
Compose allows all services, networks, and volumes to be declared in a single YAML file (dockercompose.yml). A single command can then be used to start, stop, or rebuild the entire application stack.

Why Multi-Service Applications?

Modern applications are rarely single-process systems. A typical web application requires a frontend/web server, a back-end application server, and a database, each of which may need to scale, restart, or be updated independently. Running each component as its own container, while connecting them through a shared Compose-managed network, mirrors real-world microservice architecture and simplifies development, testing, and deployment.

Key Concepts in Docker Compose

- Services — each entry under 'services:' represents one container (e.g., web, database).
- Image — the base image used to create the container for a service.
- Ports — mapping of host machine ports to container ports (HOST:CONTAINER).
- Environment Variables — used to configure containers at runtime (e.g., database credentials).
- Volumes — persistent storage that survives container removal, used here for MySQL data.
- Networks — Compose automatically creates a shared network so services can communicate using their service names as hostnames.

Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

Structure of docker-compose.yml

The Compose file follows YAML syntax and is organized under top-level keys: 'version' (Compose file format version), 'services' (list of containers to run), and 'volumes' (named volumes for persistent storage). Each service can specify its image, container name, ports, environment variables, and volume mounts.

Advantages of Docker Compose

- Single command to start/stop an entire multi-container application.
- Declarative configuration — the whole stack is defined in one version-controlled file.
- Automatic internal networking between services using service names.
- Simplified environment variable and volume management.
- Easier to reproduce identical environments across machines.

3. Setup and Configuration

The following steps describe how to set up the project directory and Compose file.

1. Open PowerShell / Command Prompt.
2. Create a new directory for the project:

```
C:\Users\203>mkdir compose-demo
```

3. Navigate into the newly created directory:

```
C:\Users\203>cd compose-demo
```

4. Open Notepad (or any text editor) to create the Compose file:

```
C:\Users\203\compose-demo>notepad docker-compose.yml
```

5. Paste the following YAML configuration into the file:

```
version: '3.8'

services:
  web:
    image: nginx
    container_name: webserver
    ports:
```

Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

```
      - "8080:80"


  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql


volumes:
  mysql-data:
```

6. Save the file. In Notepad's 'Save As' dialog, set 'Save as type' to 'All Files' and name it exactly 'docker-compose.yml' , saving it inside the compose-demo directory.
7. Verify the file was created correctly by listing directory contents:

BY COMMAND “DIR”

Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

4. **Commands Used**

Command	**Description**
`docker compose up`	Start all services defined in the Compose file
Command	**Description**
`docker compose up -d`	Start all services in detached (background) mode
`docker compose ps`	Show status of running services
`docker compose logs`	View logs of all services
`docker compose stop`	Stop running services without removing them
`docker compose start`	Restart previously stopped services
`docker compose restart`	Restart containers for all services
`docker compose down`	Stop and remove containers, networks created by up
`docker compose down -v`	Stop and remove containers, networks, and volumes
`docker compose images`	Show images used by the created containers
`docker compose config`	Validate and view the resolved Compose file

5. Procedure

The steps below were carried out in sequence, all from within the compose-demo directory in PowerShell / Command Prompt, to deploy and manage the multi-service application:

#	**Command**	**Explanation**	**Expected Output**
1	`mkdir compose-demo`	Create project directory.	Directory created.
2	`cd compose-demo`	Move into project directory.	Prompt path changes to compose-demo.

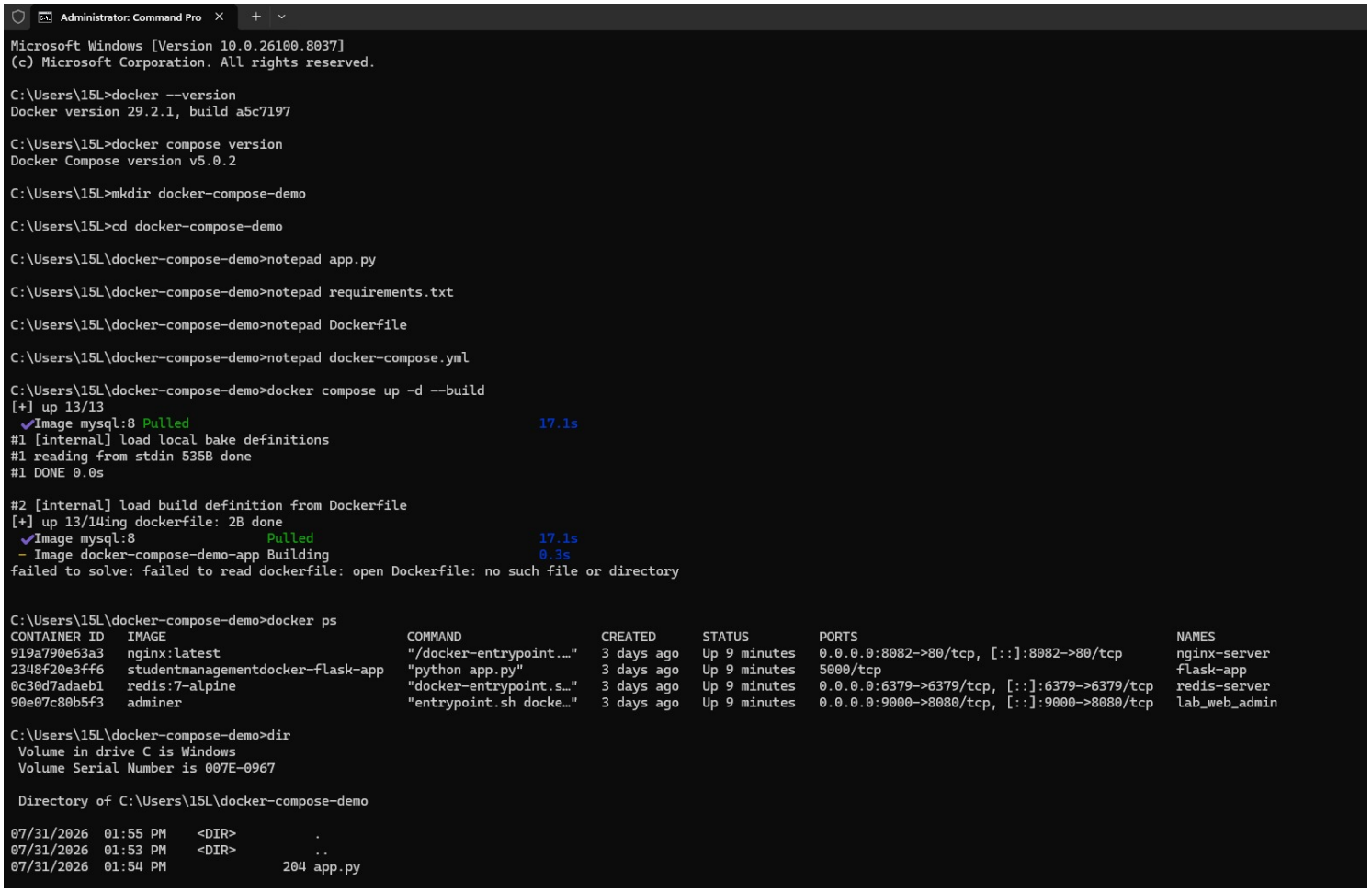
Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

3	`notepad dockercompose.yml`	Create and edit the Compose file.	Notepad opens a new file.
4	`(paste YAML, save as 'All Files')`	Define web (nginx) and database (mysql) services.	File saved as dockercompose.yml.
5	`dir`	Verify docker-compose.yml exists in the folder.	docker-compose.yml listed.
6	`docker compose config`	Validate YAML syntax before starting.	Resolved configuration printed, no errors.
7	`docker compose up -d`	Build/pull images and start both services in background.	Both containers created and started.
8	`docker compose ps`	Confirm both containers are running.	webserver and mysqlserver shown as 'running'.
#	**Command**	**Explanation**	**Expected Output**
9	`Browser -> localhost:8080`	Access the Nginx service via mapped port.	Nginx welcome page loads.
10	`docker compose logs`	View combined logs of both services.	Logs from web and database services shown.
11	`docker exec -it mysqlserver mysql -u student -p`	Connect to MySQL inside the database container.	Prompted for password; MySQL shell opens.
12	`SHOW DATABASES;`	Verify studentdb was created via environment variables.	studentdb listed among databases.
13	`docker compose stop`	Stop both running services.	Containers stop, status 'Exited'.
14	`docker compose start`	Restart the stopped services.	Containers return to 'running' state.
15	`docker compose down`	Stop and remove containers and network (volume persists).	Containers and network removed.

Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

16	`docker compose up -d`	Bring the stack back up to verify data persistence.	studentdb still present due to named volume.
17	`docker compose down -v`	Clean up completely, including the volume.	Containers, network, and volume all removed.

6. Screenshots

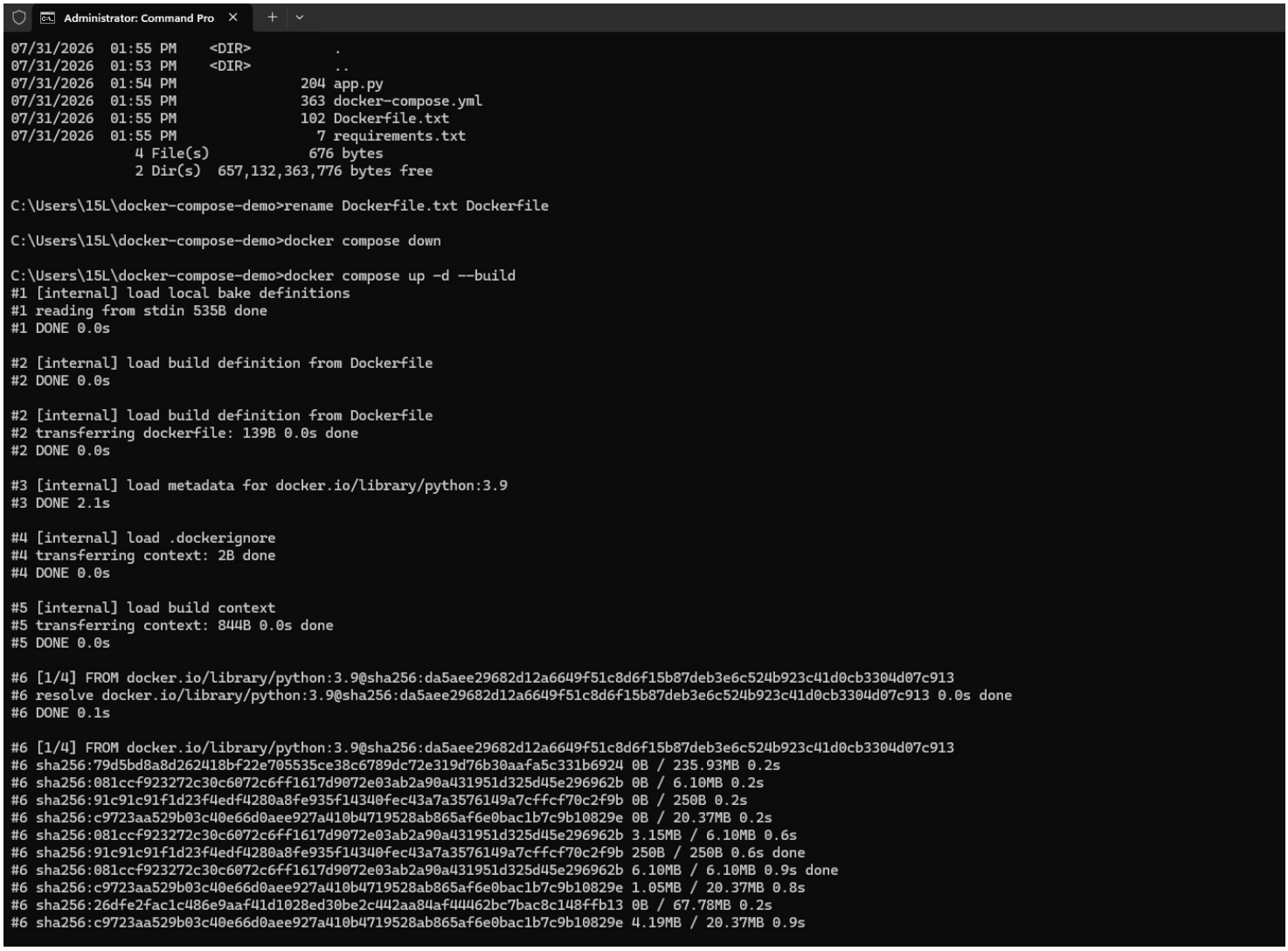


Name:- HITEN PAHILAJANI

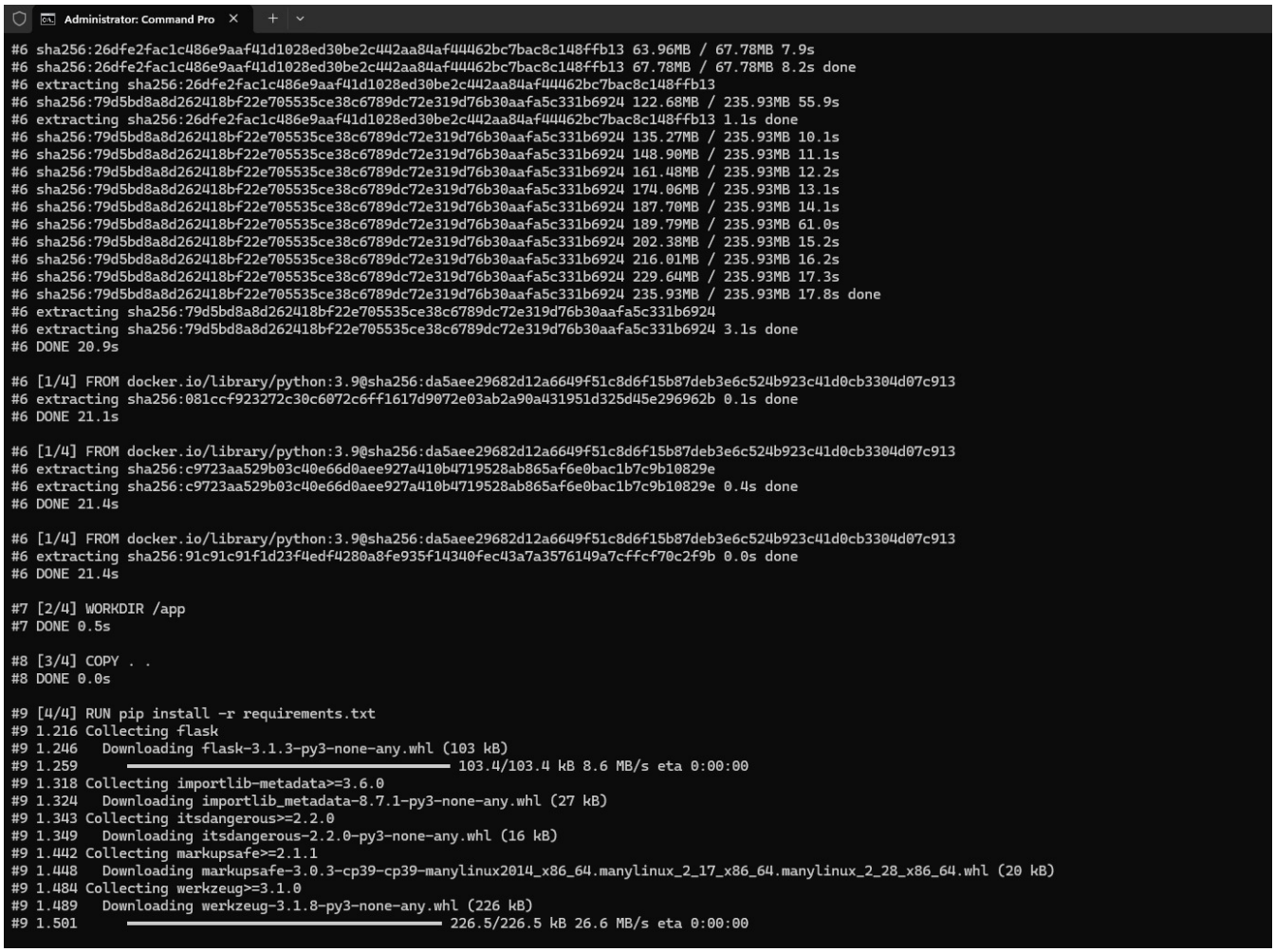
Roll No:- 2401069

Batch:- T13

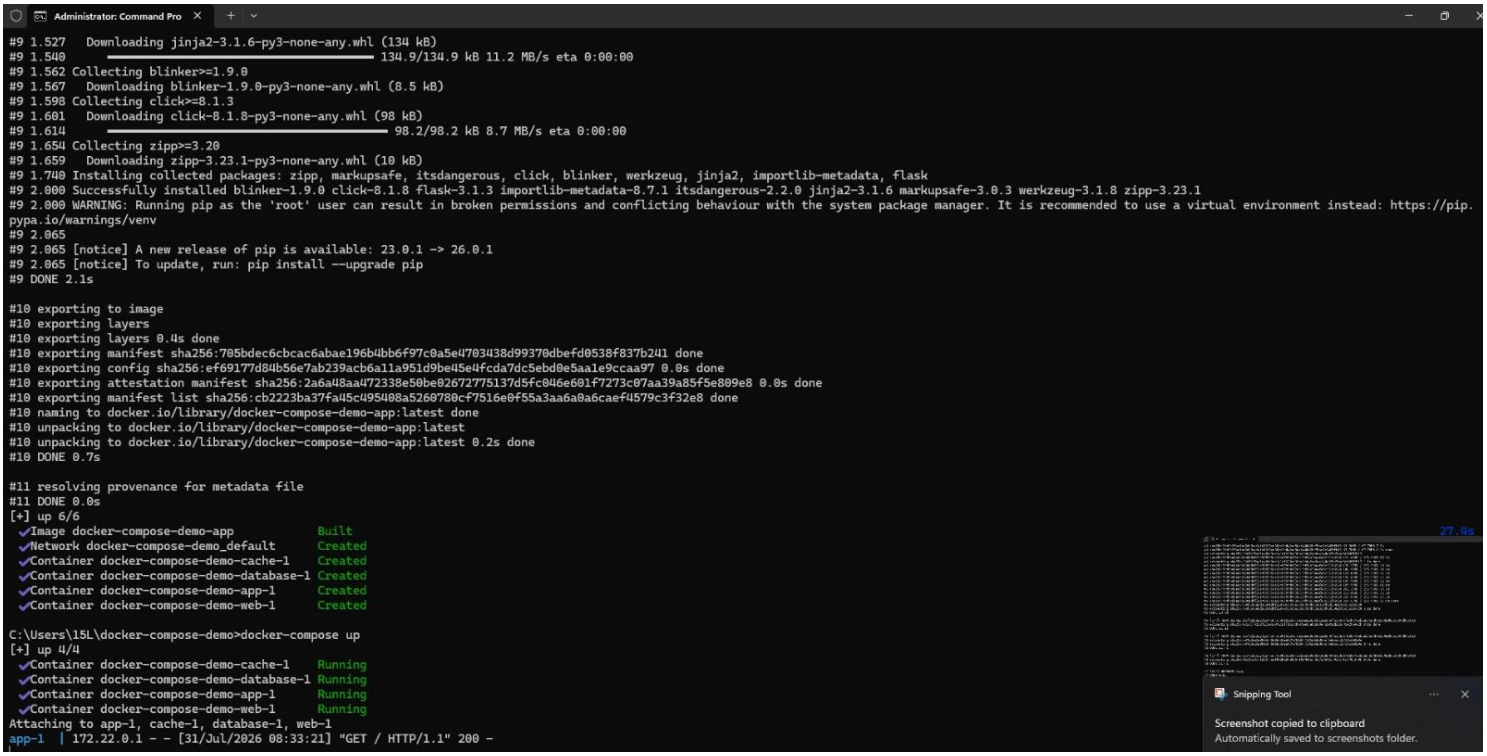
Subject : ASD&D



Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D



Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D





Name:- HITEN PAHILAJANI
Roll No:- 2401069
Batch:- T13
Subject : ASD&D

7. Observation

During this experiment, it was observed that Docker Compose simplified the deployment of a Flask web application by using a single docker-compose.yml file. The Flask application container was built from the Dockerfile and started successfully using the docker compose up command. The application was accessible through the mapped host port (localhost:5000), confirming that the container was running correctly. Docker Compose also automatically created the required network and managed the container lifecycle efficiently.

8. Result

The Flask web application was successfully deployed using Docker Compose. The container was built, started, and verified by accessing **http://localhost:5000**, where the message **"Hello from Flask inside Docker Compose!"** was displayed. The application was managed using Docker Compose commands such as docker compose up, docker compose ps, docker compose stop, and docker compose down, demonstrating successful container orchestration.

9. Conclusion

This experiment provided practical experience in deploying a Dockerized Flask application using Docker Compose. It demonstrated how a single YAML configuration file can define and manage the application service, including the build process, port mapping, and container configuration. Docker Compose simplified application deployment by automating container creation, networking, and lifecycle management. The experiment highlighted the benefits of Docker Compose in ensuring a consistent, repeatable, and efficient deployment process, making it a valuable tool for developing and managing containerized applications.